

2026 €€€ 非专业级别软件能力认证第一轮
(C5P-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 26 日 14:30~16:30

考生注意事项：

- 试题纸共有 13 页，答题纸共有 1 页，满分 100 分。本卷为自制模拟卷，不代表任何官方机构。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

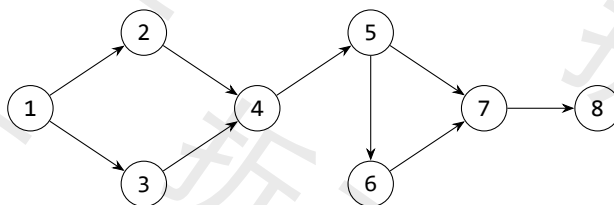
1. 下列 Linux 命令中，用于连接并显示文件内容的是（ ）
A. cp
B. cat
C. ping
D. touch
2. 已知某用二叉链表存储的哈夫曼树共有 m 个叶子结点，则该哈夫曼树中总共有多少个空指针域（ ）
A. m
B. $m+1$
C. $2m$
D. $2m+1$
3. 对于一棵包含 n 个结点的完全二叉树，对每个叶子结点向上访问直到根结点，统计路径上经过的结点数之和。总时间复杂度为（ ）
A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $O(n^2 \log n)$
4. 小 h 从原点 $(0,0)$ 出发，每一步向右或向上走，到达点 $(5,5)$ ，且始终满足 $x \geq y$ 。满足条件的路径有多少条（ ）
A. 252
B. 90
C. 132
D. 42
5. 用两个栈模拟一个队列，入队时元素统一压入栈 A。正确的出队操作是（ ）
A. B 为空时将 A 全部转移到 B，再从 B 弹出；B 非空时直接从 B 弹出
B. 无论 B 是否为空，都先将 A 全部转移到 B，再从 B 弹出

- C. 直接删除 A 的栈底元素
D. 直接从 A 弹出栈顶元素
6. 从集合 $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ 中选出 4 个不同的数, 要求任意两个被选数都不相邻。不同的选法共有 ()
- A. 15
B. 20
C. 35
D. 70
7. 在 C++ 中, 关于 `std::set<int>` 与 `std::map<int, int>` 的说法, 正确的是 ()
- A. set 中的元素可以重复
B. map 按插入顺序存储键值对
C. set 自动有序, 插入、删除、查找复杂度均为 $O(\log n)$
D. map 的键可以重复
8. 关于有向无环图的拓扑排序, 下列说法正确的是 ()
- A. 任何有向无环图的拓扑排序都唯一
B. 含有环的有向图也可以进行拓扑排序
C. 若有边 $u \rightarrow v$, 则任一拓扑序中 u 在 v 之前
D. 拓扑排序就是深度优先遍历序列
9. KMP 算法中, next 数组的主要作用是 ()
- A. 记录模式串失配后应回溯到的位置
B. 记录主串每个字符出现的次数
C. 记录主串的后缀数组
D. 与模式串无关地跳过主串位置
10. 对一个 n 个顶点、 m 条边的带权有向图使用不带堆优化的 Dijkstra 算法, 时间复杂度为 ()
- A. $O(n^2)$
B. $O(m \log n)$
C. $O((m+n) \log n)$
D. $O(mn)$
11. 已知 20260913 是质数, 表达式 $(1/2 + 1/3 + \dots + 1/20260912)^{20260913} \bmod 20260913$ 的值为 ()
- A. 0
B. 1
C. $2^{20260913} \bmod 20260913$
D. 20260912
12. 在一条长度为 2 的线段上随机取两个点, 则以这两个点为端点的线段期望长度为 ()
- A. 1
B. $1/3$

C. $2/3$

D. $3/4$

13. 下图为一个有向图。恰好删除一条边后，仍要求存在从 1 到 8 的有向路径，则可以删除的边共有（ ）条。



A. 5

B. 6

C. 7

D. 8

14. 1 到 200 之间，不能被 2、3、7 中任何一个整除的整数共有（ ）个

A. 57

B. 58

C. 59

D. 60

15. 根结点深度为 1，则一棵深度为 h 的满 k 叉树 ($k > 1$) 的结点数为（ ）

A. $(k^h - 1)/(k - 1)$

B. $(k^{(h+1)} - 1)/(k - 1)$

C. $k^h/(k - 1)$

D. k^h

二、阅读程序（判断题正确填 √，错误填 ×；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, a[N], tmp[N];
05void merge_sort(int l, int r) {
06    if (l >= r)
07        return;
08    int mid = (l + r) >> 1;
09    merge_sort(l, mid);
10    merge_sort(mid + 1, r);
11    int i = l, j = mid + 1, k = l;
12    while (i <= mid && j <= r) {
13        if (a[i] <= a[j])
14            tmp[k++] = a[i++];
15        else
16            tmp[k++] = a[j++];
17    }
18    while (i <= mid)
19        tmp[k++] = a[i++];
20    while (j <= r)
21        tmp[k++] = a[j++];
22    for (int p = l; p <= r; ++p)
23        a[p] = tmp[p];
24}
25int main() {
26    ios::sync_with_stdio(0);
27    cin.tie(0);
28    cin >> n;
29    for (int i = 1; i <= n; ++i)
30        cin >> a[i];
31    merge_sort(1, n);
32    for (int i = 1; i <= n; ++i)
33        cout << a[i] << " \n"[i == n];
```

34 }

• 判断题

16. 若初始数组为 $[4, 1, 3, 2, 5]$, 只调用 `merge_sort(1, 4)`, 调用返回后 $a[1..5]$ 为 ()
- A. $[1, 2, 3, 4, 5]$
B. $[1, 2, 4, 3, 5]$
C. $[1, 3, 2, 4, 5]$
D. $[4, 1, 3, 2, 5]$
17. 合并时若两个区间当前元素相等, 程序会先把左区间的元素写入 `tmp`。()
- A. 正确
B. 错误
18. 对任意一次已经返回的调用 `merge_sort(l, r)`, $a[1..r]$ 与 $tmp[1..r]$ 均为非降序。()
- A. 正确
B. 错误
19. 在执行 `merge_sort(1, n)` 的过程中, 同时处于调用栈中的该函数调用数量为 $O(\log n)$ 。()
- A. 正确
B. 错误

• 单选题

20. 当 $l=3, r=10$ 时, $mid=(l+r)>>1$ 后, 递归处理的两个区间是 ()
- A. $[3, 5]$ 和 $[6, 10]$
B. $[3, 6]$ 和 $[7, 10]$
C. $[3, 7]$ 和 $[8, 10]$
D. $[3, 10]$ 和空区间
21. 当 $n=8$ 且每次合并都达到比较次数的最大值时, 所有合并过程的元素比较总次数为 ()
- A. 12
B. 14
C. 17
D. 24

```
01#include <bits/stdc++.h>
02using namespace std;
03int main() {
04    ios::sync_with_stdio(0);
05    cin.tie(0);
06    int n, k;
07    string s;
```

```

08     cin >> n >> k >> s;
09     vector<char> st;
10     st.reserve(n);
11     for (char ch : s) {
12         while (k > 0 && !st.empty() && st.back() >
ch) {
13             st.pop_back();
14             --k;
15         }
16         st.push_back(ch);
17     }
18     while (k > 0) {
19         st.pop_back();
20         --k;
21     }
22     for (char ch : st)
23         cout << ch;
24 }

```

22. 当 $k=0$ 时, 程序扫描字符串的过程中不会执行 `pop_back`。()
- A. 正确
- B. 错误
23. 扫描字符 `ch` 时, 只要栈顶字符小于 `ch` 且仍可删除, 就应弹出栈顶字符。()
- A. 正确
- B. 错误
24. 扫描结束后若 $k > 0$, 从当前序列末尾继续删除字符可以得到最优结果。()
- A. 正确
- B. 错误
25. 当输入为 `8 3 dcabccba` 时, 扫描过程中 `st` 的最大长度为 ()
- A. 3
- B. 5
- C. 6
- D. 8
26. 当输入为 `8 3 dcabccba` 时, 程序的输出为 ()
- A. `abcba`
- B. `abccb`
- C. `acbba`

D. bccba

27. 若每个字符最多入栈一次、出栈一次, 则该程序的时间复杂度为 ()

A. $O(n)$

B. $O(n \log n)$

C. $O(n^2)$

D. $O(k \log n)$

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, head[N], to[N * 2], nxt[N * 2], ec;
05int dista[N], que[N];
06void add(int u, int v) {
07    to[++ec] = v;
08    nxt[ec] = head[u];
09    head[u] = ec;
10}
11int bfs(int start) {
12    fill(dista + 1, dista + n + 1, -1);
13    int l = 0, r = 0, far = start;
14    que[r++] = start;
15    dista[start] = 0;
16    while (l < r) {
17        int u = que[l++];
18        if (dista[u] > dista[far])
19            far = u;
20        for (int e = head[u]; e; e = nxt[e]) {
21            int v = to[e];
22            if (dista[v] == -1) {
23                dista[v] = dista[u] + 1;
24                que[r++] = v;
25            }
26        }
27    }
28    return far;
29}
```

```

30 int main() {
31     ios::sync_with_stdio(0);
32     cin.tie(0);
33     cin >> n;
34     for (int i = 1, u, v; i < n; ++i) {
35         cin >> u >> v;
36         add(u, v);
37         add(v, u);
38     }
39     int s = bfs(1);
40     int t = bfs(s);
41     cout << dista[t];
42 }

```

28. 对输入边依次为 1-2,1-3,2-4,2-5,3-6,6-7 的树, 调用 bfs(1) 返回的结点是 ()
- A. 4
B. 5
C. 6
D. 7
29. 若 BFS 过程中存在多个结点与当前最远距离相同, 程序保留最先从队列中取出的那个结点。()
- A. 正确
B. 错误
30. fill(dista+1,dista+n+1,-1) 不会初始化 dista[0], 但不影响程序对输入结点的处理。()
- A. 正确
B. 错误
31. 函数 bfs 中, 队列尾指针入队一个新结点时应执行 ()
- A. que[r] = v;
B. que[r++] = v;
C. que[++l] = v;
D. que[l++] = v;
32. 数组 to[N*2] 至少可以容纳任意一棵满足上界的树所产生的全部邻接项。()
- A. 正确
B. 错误
33. (本题 4 分) 若输入的树有边 1-2,1-3,2-4,2-5,3-6,6-7, 程序输出为 ()
- A. 3

B. 4

C. 5

D. 6

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (有序序列的第 k 小和) 有两个长度均为 n 的单调不降序列 a 和 b 。从两个序列中各取一个数相加，共得到 n^2 个和，求这些和中第 k 小的值。输入满足 $1 \leq n \leq 10^5$, $1 \leq k \leq n^2$ ，元素为非负整数且答案不超过 10^9 。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 100005;
04int n, a[N], b[N];
05long long k;
06int upper_bound_index(int *a, int n, long long x) {
07    int l = 0, r = ①;
08    while (l < r) {
09        int mid = (l + r) >> 1;
10        if (②)
11            r = mid;
12        else
13            l = mid + 1;
14    }
15    return ③;
16}
17long long count_le(long long sum) {
18    long long cnt = 0;
19    for (int i = 0; i < n; ++i)
20        cnt += upper_bound_index(b, n, sum - a[i]);
21    return cnt;
22}
23long long solve() {
24    long long l = ④, r = ⑤;
25    while (l < r) {
26        long long mid = (l + r) >> 1;
27        if (count_le(mid) >= k)
28            r = mid;
29        else
30            l = mid + 1;
31    }
```

```

32     return l;
33 }
34 int main() {
35     ios::sync_with_stdio(0);
36     cin.tie(0);
37     cin >> n >> k;
38     for (int i = 0; i < n; ++i) cin >> a[i];
39     for (int i = 0; i < n; ++i) cin >> b[i];
40     cout << solve();
41 }

```

34. ①处应填 ()
 A. n B. n-1
 C. x D. 0
35. ②处应填 ()
 A. $a[mid] > x$ B. $a[mid] \geq x$
 C. $a[mid] < x$ D. $a[mid] \leq x$
36. ③处应填 ()
 A. l B. l+1
 C. r-1 D. n-1
37. ④处应填 ()
 A. $a[0]+b[0]$ B. $a[1]+b[1]$
 C. 0 D. $a[n]+b[n]$
38. ⑤处应填 ()
 A. $a[n-1]+b[n-1]$ B. $a[n]+b[n]$
 C. $2*N$ D. n-1

(2) (最小生成树) 给定一个有 n 个结点、 m 条带权无向边的图, 求其最小生成树的边权和; 若图不连通则输出 Impossible。输入满足 $1 \leq n \leq 2 * 10^5$, $1 \leq m \leq 4 * 10^5$ 。

```

01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 2e5 + 5, M = 4e5 + 5;
04 struct Edge {
05     int u, v, w;
06     bool operator < (const Edge &other) const {
07         return w < other.w;
08     }

```

```

09} e[M];
10int n, m, fa[N];
11int find(int x) {
12    return fa[x] == x ? x : fa[x] = find(fa[x]);
13}
14int main() {
15    ios::sync_with_stdio(0);
16    cin.tie(0);
17    cin >> n >> m;
18    for (int i = 1; i <= n; ++i)
19        fa[i] = ①;
20    for (int i = 0; i < m; ++i)
21        cin >> e[i].u >> e[i].v >> e[i].w;
22    sort(e, e + ②);
23    long long ans = 0;
24    int used = 0;
25    for (int i = 0; i < m; ++i) {
26        int x = find(e[i].u), y = find(e[i].v);
27        if (③)
28            continue;
29        fa[x] = ④;
30        ans += e[i].w;
31        ++used;
32    }
33    if (used != ⑤)
34        cout << "Impossible";
35    else
36        cout << ans;
37}

```

39. ①处应填 ()

- A. i B. 0
C. n D. fa[i-1]

40. ②处应填 ()

- A. m B. m-1
C. M D. n

41. ③处应填 ()

- A. `x == y` B. `x != y`
C. `x < y` D. `e[i].u == e[i].v`

42. ④处应填 ()

- A. `x` B. `y`
C. `e[i].v` D. `fa[y]`

43. ⑤处应填 ()

- A. `n` B. `n-1`
C. `m` D. `used+1`